LangGraph

Stateful, graph-based agent orchestration by LangChain

1. What is LangGraph?

LangGraph is an open-source framework by LangChain for building stateful AI agents and multi-agent systems. It models your agent workflow as a directed graph where nodes are functions (agent steps) and edges define how control flows between them.

The big difference from a simple LangChain chain is that LangGraph supports loops. Chains are linear — input goes in, output comes out. LangGraph graphs can cycle back, meaning an agent can keep working, calling tools, reflecting, and retrying until it actually finishes the job.

> **Why it matters:** Real agent tasks are not linear. An agent needs to check its work, handle errors, call multiple tools in sequence, and sometimes wait for a human to approve something. LangGraph is built for exactly that.

2. The Three Core Concepts

State — the shared memory of the graph

State is a TypedDict (a typed Python dictionary) that every node in the graph can read and write. It is the single source of truth as execution flows through the graph. When a node runs, it receives the current state and returns a dict of updates to apply to it.

```
from typing import TypedDict, Annotated
from langgraph.graph import StateGraph
import operator

class AgentState(TypedDict):
    messages: Annotated[list, operator.add]  # list that appends
    user_query: str
    search_results: str
    final_answer: str
```

The Annotated[list, operator.add] syntax tells LangGraph how to merge updates from nodes — in this case, append new messages to the list rather than overwrite it. You can define custom reducers for any field.

Nodes — the workers

A node is just a Python function. It takes the current state as input and returns a dict of the fields it wants to update. You can put any logic inside a node — LLM calls, tool execution, database queries, anything.

```
from langchain_google_genai import ChatGoogleGenerativeAI

llm = ChatGoogleGenerativeAI(model='gemini-2.0-flash')

def call_llm(state: AgentState) -> dict:
    response = llm.invoke(state['messages'])
    return {'messages': [response]}   # appended via reducer

def run_search(state: AgentState) -> dict:
    results = search_tool(state['user_query'])
    return {'search_results': results}
```

Edges — how control moves

Edges connect nodes and tell LangGraph where to go next. There are two types:

- **Direct edge:** always goes from node A to node B, no conditions.
- **Conditional edge:** a routing function reads the current state and returns the name of the next node to go to. This is how you build branching logic and loops.

```
from langgraph.graph import END

def should_continue(state: AgentState) -> str:
    last_msg = state["messages"][-1]
    if hasattr(last_msg, 'tool_calls') and last_msg.tool_calls:
        return "run_tools"     # go to tool node
    return END                 # done, exit the graph
```

> **Key insight:** The routing function is just Python — it reads state and returns a string (node name or END). No magic. This is what makes LangGraph so flexible for complex conditional workflows.

---

3. Building a Graph — Full Example

Here is a minimal but complete agent that calls an LLM, routes to tools if needed, and loops until done:

```
from langgraph.graph import StateGraph, END
from langgraph.prebuilt import ToolNode

# 1. Define the graph with your state schema
graph = StateGraph(AgentState)

# 2. Add nodes
graph.add_node('llm', call_llm)
graph.add_node('tools', ToolNode(tools=[search_tool]))

# 3. Set the entry point
graph.set_entry_point('llm')

# 4. Add edges
graph.add_conditional_edges(
    'llm',
    should_continue,
    {'run_tools': 'tools', END: END}
)
graph.add_edge('tools', 'llm')  # after tools, always go back to llm

# 5. Compile and run
app = graph.compile()
result = app.invoke({'messages': [HumanMessage('What is the weather in London?')],
                     'user_query': 'weather London'})
```

The flow here: LLM node runs → sees a tool call → routes to tools node → tools node runs the search → routes back to LLM → LLM sees the result and writes the final answer → routes to END.

---

4. Key Features

Checkpointing — persistence and time-travel

LangGraph can save the full graph state after every node execution. This is called a checkpoint. It means:

- If the agent crashes mid-run, you can resume from the last checkpoint instead of starting over
- You can replay any past execution step by step — useful for debugging
- You can fork from a past state and try a different path (time-travel)

```
from langgraph.checkpoint.memory import MemorySaver   # dev/testing
from langgraph.checkpoint.postgres import PostgresSaver  # production

checkpointer = MemorySaver()
app = graph.compile(checkpointer=checkpointer)

# Each run needs a thread_id so checkpoints are scoped to a session
config = {'configurable': {'thread_id': 'user_123_session_1'}}
app.invoke(initial_state, config=config)
```

Human-in-the-Loop — pause and resume

LangGraph can pause a running graph at a specific node and wait for a human to review or approve before continuing. This is done with interrupt_before or interrupt_after when compiling the graph.

```
app = graph.compile(
    checkpointer=checkpointer,
    interrupt_before=['run_tools']   # pause before tools run
)

# First call — runs until the interrupt
app.invoke(state, config=config)

# Human reviews here... then resumes with None input
app.invoke(None, config=config)
```

Because checkpointing saves state, the graph picks up exactly where it left off after the human approves. No data is lost.

Subgraphs — modular multi-agent systems

You can nest graphs inside other graphs. A subgraph is a compiled LangGraph that gets used as a single node inside a parent graph. This is how you build large multi-agent systems in a clean, modular way.

- Each subgraph has its own state schema and its own nodes/edges
- The parent graph passes data into the subgraph and receives its output — it does not know or care what happens inside
- Example: a parent orchestrator graph delegates to a ResearchSubgraph and a WritingSubgraph as separate nodes

```
research_graph = StateGraph(ResearchState)
# ... define research nodes and edges ...
research_app = research_graph.compile()

# Use the compiled subgraph as a node in the parent
parent_graph = StateGraph(OrchestratorState)
parent_graph.add_node('research', research_app)
```

5. LangGraph vs Other Frameworks

Feature	LangGraph	CrewAI	AutoGen
Mental model	Graph of nodes and edges	Team of role-based agents	Group chat between agents
State management	Typed shared state dict	Task outputs passed along	Conversation message history
Loops and cycles	Native, first-class	Limited	Via GroupChat turns
Checkpointing	Built-in (Postgres, SQLite)	Not built-in	Not built-in
Human-in-the-loop	Built-in interrupt gates	Manual	Via HumanProxyAgent
Control granularity	Very high — full control	Medium — role-based	Medium — message-based
Setup complexity	Medium-High	Low	Low-Medium
Best for	Complex production workflows	Quick role-based setups	Research, multi-agent debate

6. When to Use LangGraph

- **You need an agent that loops —** calls tools, checks results, retries, until done
- **You need stateful multi-turn conversations —** ere context must persist across steps
- **You need human-in-the-loop —** ause execution, get approval, then continue
- **You need production-grade persistence —** gent survives restarts, supports resume
- **You need fine-grained control —** er routing logic that role-based frameworks cannot express
- **You are building a supervisor/multi-agent system —** d want clean subgraph modularity

> **When NOT to use it:** If your workflow is simple and linear (input → one LLM call → output), LangGraph is overkill. Use a plain LangChain chain or even just a direct API call instead.

7. Quick Reference

Concept	What it is	How to use
StateGraph	The graph object you build your workflow on	graph = StateGraph(YourState)
State (TypedDict)	Shared memory — all nodes read and update this	class S(TypedDict): field: type
Node	A Python function that takes state and returns updates	graph.add_node('name', fn)
Direct edge	Always goes A → B	graph.add_edge('A', 'B')
Conditional edge	Routing fn picks the next node	graph.add_conditional_edges(...)
END	Special token that terminates the graph	return END from routing function

Concept	What it is	How to use
ToolNode	Pre-built node that executes tool calls from messages	from langgraph.prebuilt import ToolNode
MemorySaver	In-memory checkpointer for dev/testing	graph.compile(checkpointer=MemorySaver())
PostgresSaver	Production checkpointer backed by Postgres	graph.compile(checkpointer=PostgresSaver(...))
interrupt_before	Pause the graph before a named node runs	graph.compile(interrupt_before=['node'])
Subgraph	A compiled graph used as a node inside another graph	parent.add_node('sub', sub_app)